

Document number: P1401R5
Date: 2021-04-12
Audience: CWG
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Narrowing contextual conversions to bool

This paper proposes to allow conversions from integral types to type `bool` in `static_asserts` and `if constexpr`-statements.

Tony Tables

Today	If accepted
<code>if constexpr(bool(flags & Flags::Exec))</code>	<code>if constexpr(flags & Flags::Exec)</code>
<code>if constexpr((flags & Flags::Exec) != 0)</code>	<code>if constexpr(flags & Flags::Exec)</code>
<code>static_assert(bool(N));</code>	<code>static_assert(N);</code>
<code>static_assert(N % 4 != 0);</code>	<code>static_assert(N % 4);</code>

Revisions

R4 → R5

Updated wording as per CWG instructions.

R3 → R4

Updated wording by adding examples, as instructed by EWG.

R2 → R3

Updated references to match the latest Standard Draft.

Updated the table showing compiler implementation to match the most recent compiler versions.

Added more discussion and background information that might help EWG make a more informed decision:

- described pedantic mode,
- discussed the question of how often users report this as a problem,
- proposed leveraging warnings instead of constraining the language,
- described vendor view on implementing the no-narrowing requirement.

R1 → R2

Incorporated feedback from Richard Smith (the submitter of [\[CWG 2039\]](#)). Added recommended resolution and wording.

R0 → R1

Extended the discussion and problem analysis. Outlined the reange of possible changes. Not proposing wording anymore: the goal is to obtain the direction from EWG first.

Motivation

Clang currently fails to compile the following program, and this behavior is Standard-compliant:

```
enum Flags { Write = 1, Read = 2, Exec = 4 };

template <Flags flags>
int f() {
    if constexpr (flags & Flags::Exec) // fails to compile due to narrowing
        return 0;
    else
        return 1;
}

int main() {
```

```
    return f<Flags::Exec>(); // when instantiated like this
}
```

This is because, as currently specified, narrowing is not allowed in contextual conversion to `bool` in core constant expressions. If compilers were standard-compliant, even the following code would be ill-formed.

```
template <std::size_t N>
class Array
{
    static_assert(N, "no 0-size Arrays");
    // ...
};

Array<16> a; // fails to compile in pure C++
```

All these situations can be fixed by applying a `static_cast` to type `bool` or comparing the result to 0, but the fact remains that this behavior is surprising. For instance, run-time equivalents of the above constructs compile and execute fine:

```
if (flags & Flags::Exec) // ok
{}

assert(N); // ok
```

Note that the proposal only affects the contextual conversions to `bool`: it does not affect implicit conversions to `bool` in other contexts.

This paper addresses issue [\[CWG 2320\]](#).

Background

The no-narrowing requirement was added in [\[CWG 2039\]](#), which indicates it was intentional. However, the issue documentation does not state the reason.

The no-narrowing requirement looks justified in the context of `noexcept` specifications, where the "double `noexcept`" syntax is so strange that it can be easily misused. For instance, if I want to say that my function `f()` has the same `noexcept` specification as function `g()`, it doesn't seem impossible that I could mistakenly type this as:

```
void f() noexcept(g);
```

To the uninitiated this looks reasonable; and it compiles. Also, if `g()` is a `constexpr` function, the following works as well:

```
void f() noexcept(g());
```

The no-narrowing requirement helps minimize these bugs, so it has merit. But other contexts, like `static_assert`, are only victims thereof.

Analysis

Affected contexts

The definition of *contextually converted constant expression of type `bool`* ([\[expr.const/10\]](#)) is used in four places in the standard:

- `static_assert`
- `if constexpr`
- `explicit(bool)`
- `noexcept(bool)`

Note that `requires`-clause does not use the definition, as it requires that the expression "shall be a constant expression of type `bool`" ([\[temp.constr.atomic/3\]](#)). The problems caused by the contextually converted constant expression of type `bool` are mostly visible in the first two cases. In case of `explicit(bool)` we expect a type trait to be used as an expression.

Similarly, in the case of `noexcept(bool)` we only expect a type trait or a `noexcept`-expression.

The following table illustrates where compilers allow a conversion to `bool` in a contextually converted constant expression of type `bool` against the C++ requirements:

context	gcc 10.1	clang 10.0.0	icc 19.0.1	msvc 19.24
<code>static_assert</code>	yes	yes	yes	yes
<code>if constexpr</code>	yes	no	yes	yes
<code>explicit(bool)</code>	no	no	--*	yes

<code>noexcept(bool)</code>	no	yes	yes	yes
-----------------------------	----	-----	-----	-----

* Feature not implemented in this compiler.

Accepting this proposal would be to some extent standardizing the existing practice among compiler vendors.

Types contextually convertible to type `bool`

The following table lists types that are contextually convertible to type `bool`:

Type	Allowed in constant expr	true when
class with conversion to <code>bool</code>	yes	operator returns <code>true</code>
class with conversion to a built-in type convertible to <code>bool</code>	as per rules below	as per below rules
object/function pointer	no	not null
function name/reference	no	always
array name/reference	no	always
pointer to member	no	not null
integral type	no, except for 0 and 1	not zero
floating-point type	no	not (plus or minus) zero
<code>nullptr_t</code>	no	never
unscoped enumeration	no	not zero

The problem, which this proposal is trying to fix, has only been reported when conversions from integral types or unscoped enumeration types are involved, as for these types such conversion has practical and often used meaning:

- non-empty collection/sequence,
- is a given bit in a bitmask set.

We have never encountered a need to check if a floating-point value is exactly ± 0 in this way. Technically, checking a pointer has a meaning: "is it really pointing to some object/function", but it is more difficult to imagine a practical use case for it in *contextually converted constant expression of type `bool`*.

Implicit conversions to `bool`

Some have suggested that a conversion to `bool` in general should not be considered narrowing, that `bool` should not be treated as a small integral type, and that the conversion to `bool` should be treated as a request to classify the states of the object as one of the two categories.

We do not want to go there. Even this seemingly correct reasoning can lead to bugs like this one:

```
struct Container {
    explicit Container(bool zeroSize, bool zeroCapacity) {}
    explicit Container(size_t size, int* begin) {}
};

int main() {
    std::vector<int> v;
    X x (v.data(), v.size()); // bad order!
}
```

If the feature that prevents narrowing conversions can detect this bug, we would like to use this opportunity.

Implicit conversions to `bool` in constant expressions

Another situation brought up while discussing this problem was if the following code should be correct:

```
// a C++03 type trait
template <typename T>
struct is_small {
    enum { value = (sizeof(T) == 1) };
};

template <bool small> struct S {};
S<is_small<char>::value> s; // int 1 converted to bool
```

In constant expressions the situation is different, because whether a conversion is narrowing or not depends not only on the types but also on the values, which are known at compile-time. We think that after [P0907r4](#) was adopted, and `bool` has a well defined representation, conversion from 1 to true is now defined as non-narrowing.

Compatibility with C `assert()` macro

As described in [\[LWG 3011\]](#), currently macro `assert()` from the C Standard Library only allows expressions of scalar types, and does not support the concept of expressions "contextually converted to `bool`". We believe that this issue does not interact with our proposal. For instance, the following example works even under the C definition of macro `assert()`:

```
template <std::size_t N>
auto makeArray()
{
    assert(N);
    // ...
}
```

But it stops working if we change `assert(N)` to `static_assert(N)`.

Feedback from Richard Smith

Richard Smith — the submitter of [\[CWG 2039\]](#) — points out that the original defect report only suggested the modification to `noexcept(bool)` context. It suggested wording improvements for `static_assert` context, but they were not supposed to alter the semantics. Thus, the changes to `static_assert` semantics got there against the submitters' intentions. (The other two contexts — `if constexpr` and `explicit(bool)` — were not in the Standard at that time.) Also, the recommendation from Richard is to apply contextually converted constant expression of type `bool` only to `noexcept(bool)` and `explicit(bool)` contexts.

Options

There is a two-dimensional space of possible solutions to this problem with two extremal solutions being:

1. Leave the specification as it is: no narrowing is allowed. (This leaves all the known compilers non-conformant.)
2. Just allow any implicit conversions in contextually converted constant expression of type `bool`. (This compromises the solution in [\[CWG 2039\]](#), whatever its intent was.)

The two "degrees of freedom" in the solution space are:

1. Apply the fix only in the subset of the four contexts where the definition of contextually converted constant expression of type `bool` is used; e.g., only in `static_assert` and `if constexpr`.
2. Allow conversion to `bool` only from a subset of types implicitly convertible to `bool`, e.g., only integral and scoped enumeration types.

In fact, relaxing only `static_assert` and `if constexpr`, only for integral and scoped enumeration types solves all issues that have been reported by users that we are aware of.

Cost-benefit analysis

In general, the argument for preventing the narrowing is to prevent more bugs, whereas the argument for allowing the narrowing is to allow more expressibility and a sense of uniformity between language constructs. In this section we expand on these arguments.

Enforcing good practices

The four contexts that make use of the contextually converted constant expression of type `bool` are relatively new, at least compared to constructs inherited from C that use implicit conversions to `bool`. This could be seen as an opportunity to partially fix the past decisions that are perceived as wrong to allow narrowing conversion in run-time `if`-statements and in C `assert()`s. "We cannot change the past, so at least let's fix the thing in the new places where breaking backwards compatibility will be manageable."

The type of error that would be prevented is when function invocation is confused with function address. For instance, I wanted to call `if(is_small())` but I omitted parentheses and I got `if(is_small)` which still compiles, but now has unintended semantics.

It is also true that if the programmer is forced to provide an expression that is of type `bool`, the code is easier to understand. Consider:

```
int test(int x, int y)
{
    if (x + y)
        // ...

    if (x == -y)
        // ...
}
```

The expression in the second `if`-statement expresses the intent more clearly.

Consistency of the language

C++ language has already developed the balance between explicitness and expressiveness. It is mainly inherited from C. The following code is easily understood by C++ programmers.

```
assert (state & READ_FLAG);
```

Keeping this work and at the same preventing

```
static_assert (state & READ_FLAG);
```

Will make the programmers ask, why is this language so inconsistent? Why am I first told to learn about the implicit conversions to `bool` only to later find that they do not work in similar contexts?

Using other tools for enforcing best practices

Enforcing a certain explicit, safe programming style is important, especially in bigger teams. However, this does not have to be settled directly in the language. A common practice in such cases is to use a separate static analyzer. A characteristic property of such tools is that any of dozens of unsafe features is defined separately. A user makes the decision which unsafe features to detect and prevent, and which to allow as harmless. For instance, Clangs static analyzer `clang-tidy` has check [readability-implicit-bool-conversion](#). Using this mechanism, the leader of development team can decide which checks to enable, and this way define a safe language subset that works for the given project. Not only does it give a better control of programming constraint trade-offs, but also does not penalize other programmers that favor expressiveness over explicitness.

For a similar reason, the bug-prone decision in C++ that class constructors by default can be used for implicit conversions is becoming an insignificant issue, because present static analyzers make it easy for programmers to statically detect bugs caused by this behavior. Anyone can enable or define a check like this, and forget the explanations about preserving backwards compatibility.

Similarly, in the case of contextually converted constant expression of type `bool`, rather than introducing an inconsistent constraint against the current implementation practice, we could rely on static analyzers. We could even think about coming with a list of recommendations for compiler and tool writers, which usages of the language are considered "unsafe", in order to encourage warnings to be reported. We could even go further and introduce a new quality in the Standard: that implementations should emit a diagnostic message in certain cases for a well defined program.

How often users complain about this?

A question was asked, how often users complain about this. We believe that this is the wrong question. Given the two contexts where this is really relevant — `static_assert` and `if constexpr` — compilers GCC, MSVC and ICC simply do not adhere to the Standard and allow the narrowing. So there is no reason why users would complain. None of the four compilers listed earlier even attempts to implement `static_assert` in the strictly compliant way. This is how vendors avoid disappointing the users.

The only compiler that could record any complaints is Clang: it allows narrowing in `static_assert` but prevents it in `if constexpr`. But even here the motivation for filing a bug is small. People want to get their job done. If they face the choice between just wrapping their expression in a C-style cast to `bool` and setting up their account in LLVM's bug tracking system and filing all the forms, they will likely choose the former.

Pedantic mode

Compilers gcc, icc and clang provide a *pedantic* mode that eagerly detect any nonconformant code that would be otherwise allowed by the compiler for programmer's convenience. In gcc and clang the compiler switch for this mode is `-Wpedantic`, in icc it is `-pedantic`. Interestingly, none of the three compilers warns in pedantic mode about this narrowing conversion to `bool` in the contexts that they choose to support in a non-conformant way.

MSVC, which does not appear to have a pedantic mode, does not warn about the narrowing conversion to `bool` in all four contexts even in the highest warning level `/WX`.

Programmers often use pedantic mode to learn which constructs they use are Standard-compliant. In this sense, compilers reaffirm the belief that putting an `int` inside `static_assert` is Standard-compliant, and they have been doing this for 9 years now.

Can implementers afford to implement the constraints?

If WG21 were to reinforce the decision to prevent narrowing in `static_assert` and `if constexpr` it would require of implementers to apply a backwards incompatible change and potentially break their users' code. The author has asked four vendors — GCC, Clang, ICC and MSVC — if they would be willing to implement it this is the decision in EWG.

All of them gave the same answer. They would. This would require a two-step process. First, add the constraint in pedantic mode. This way the users can check ahead of time what code will break in the future, and prepare themselves for the impact. Next, introduce the constraint in the default mode, but still allow the conversion in "permissive" mode, so that the users who cannot or will not change their code can still use the newer versions of the compiler.

Our recommendation

We propose to apply the contextually converted constant expression of type `bool` restriction only in function declarations, that is in `noexcept(bool)` and `explicit(bool)` contexts.

In statements (`static_assert` and `if constexpr`) any conversion to `bool` should be allowed.

The distinction is clear: function declarations, which constitute an interface, require an additional caution and we require the expressions to be more restricted. We also do not expect expressions other than type traits and `noexcept()` operator. In contrast, in statements that are part of function implementation details we allow more liberty which is compatible with 'runtime equivalents' such as `if`-statements and C-asserts.

Proposed wording changes

The propose wording is a delta from [N4861](#).

Insert the following example after [\[except.spec\].para 2](#).

[Example:

```
void f() noexcept(sizeof(char[2])); // error: narrowing conversion of value 2 to type bool  
void g() noexcept(sizeof(char)); // OK, conversion of value 1 to type bool is non-narrowing
```

— end example]

Insert the following example after [\[dcl.fct.spec\].para 4](#).

[Example:

```
struct S {  
  explicit(sizeof(char[2])) S(char); // error: narrowing conversion of value 2 to type bool  
  explicit(sizeof(char)) S(bool); // OK, conversion of value 1 to type bool is non-narrowing  
};
```

— end example]

Change [\[dcl.dcl\].paragraph 6](#) as follows.

In a `static_assert`-declaration, the constant-expression ~~shall be a contextually converted constant expression of type `bool` (7.7)~~ is contextually converted to `bool` and the converted expression shall be a constant expression (7.7). If the value of the expression when so converted is `true`, the declaration has no effect. Otherwise, the program is ill-formed, and the resulting diagnostic message (4.1) shall include the text of the *string-literal*, if one is supplied, except that characters not in the basic source character set (5.3) are not required to appear in the diagnostic message.

[Example:

```
static_assert(sizeof(int) == sizeof(void*), "wrong pointer size");  
static_assert(sizeof(int[2]); // OK, narrowing allowed
```

— end example]

Change the beginning of [\[stmt.if\].paragraph 2](#) as follows.

If the `if` statement is of the form `if constexpr`, the value of the condition ~~shall be a contextually converted constant expression of type `bool` (7.7)~~ is contextually converted to `bool` and the converted expression shall be a constant expression (7.7); this form is called a `constexpr if` statement. If the value of the converted condition is false, the first substatement is a discarded statement, otherwise the second substatement, if present, is a discarded statement. During the instantiation of an enclosing templated entity ([temp.pre]), if the condition is not value-dependent after its instantiation, the discarded substatement (if any) is not instantiated.

[Example:

```
if constexpr (sizeof(int[2])) // OK, narrowing allowed  
{ }
```

— end example]

Acknowledgements

Jens Maurer reviewed the wording and offered useful suggestions.

Jason Merrill originally reported this issue in CWG reflector. Tomasz Kamiński reviewed the paper and suggested improvements.

Members of EWGI significantly improved the quality of the paper.

References

- [CWG 2039] — Richard Smith, "Constant conversions to `bool`" (http://www.open-std.org/jtc1/sc22/wg21/docs/cwg_defects.html#2039).
- [CWG 2320] — Richard Smith, "`constexpr if` and boolean conversions" (http://wiki.edg.com/pub/Wg21summer2020/CoreWorkingGroup/cwg_closed.html#2320).
- [LWG 3011] — Jonathan Wakely, "Requirements for `assert(E)` inconsistent with C" (<https://cplusplus.github.io/LWG/issue3011>).
- [WG14 N2207] — Martin Sebor, Jonathan Wakely, Florian Weimer, "Assert Expression Problematic For C++" (<http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2207.htm>).
- [P0907r4] — JF Bastien, "Signed Integers are Two's Complement" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0907r4.html>).
- [CLANG-TIDY] — Extra Clang Tools 11 documentation (<https://clang.llvm.org/extra/clang-tidy/checks/readability-implicit-bool-conversion.html>)